

摘要：

Anthropic发布Claude Opus 5，性能接近旗舰Fable 5，价格直接砍半。更颠覆认知的是：他们将Claude Code系统提示词删减80%以上，编码评测分数丝毫未降。模型越聪明，规则反成枷锁——Anthropic由此提炼出"上下文工程"新方法论，6条旧经验全面推翻。从"给规则"到"给判断空间"，AI协作范式正在悄然换挡。

Anthropic 发布了 Claude Opus 5，接近 Claude Fable 5 的前沿智能，价格减半。测评成绩上大幅超过 Opus 4.8，甚至许多测评高于 Fable 5。

Introducing Claude Opus 5

2026年7月24日

Anthropic 同时公布了一个惊人发现，模型够聪明之后，你写的那些规矩，反而在拖后腿。Claude Code 的系统提示词，砍掉了80%以上，编码评测分数则完全不变。



The new rules of context engineering for Claude 5 generation models

We removed over 80% of Claude Code's system prompt for more advanced models. How to apply the lessons we learned to your own context engineering in Claude Code and with your own agents.

Category
Claude Code Agents

Product
Claude Code
Claude Enterprise
Claude Platform

Date
July 24, 2026

Reading time
5 min

Share
Copy link

Anthropic 团队从这次“大瘦身”里总结出了上下文工程（Context Engineering）新方法论，6条旧经验被推翻，并给出了对应的替代方案。不管你是用 Claude Code 写代码，还是自己搭 Agent，都值得细读。

半价的旗舰

Opus 5 接近 Claude Fable 5 的前沿智能，价格减半。

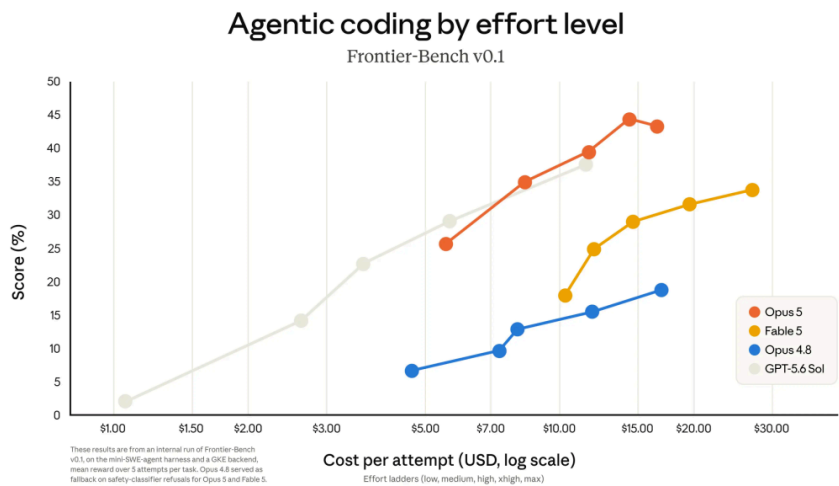
	Opus 5	Fable 5	Opus 4.8	GPT-5.6 Sol
Agentic terminal coding Frontier-Bench v0.1	43.3%	33.7%	21.1%	34.4%
Knowledge work GDPval-AA v2	1861	1747	1593	1736
Novel problem-solving ARC-AGI-3	30.2%	—	1.5%	7.8%
Agentic search BrowseComp	90.8%	87.4%	84.3%	90.4%
Multidisciplinary reasoning Humanity's Last Exam	56.3% no tools	56.5% no tools	49.8% no tools	—
	64.7% with tools	63.9% with tools	57.9% with tools	—
Computer use OSWorld 2.0	70.6%	66.1%	55.7%	62.6%
Agentic coding DeepSWE v1.1	68.8%	69.7%	59.0%	72.7%
Agentic coding FrontierCode v1.1, Main	53.4%	53.5%	46.5%	47.5%
Business workflows AutomationBench	26.0%	17.4%	17.0%	18.1%
Legal Legal Agent Benchmark, Held-out	11.7%	13.3%	10.4%	2.5%
Health HealthBench Professional	59.8%	Mythos 5 66.0%	57.4%	60.5%
Biology BioMysteryBench	49.4% hard	46.5% hard	42.4% hard	—
	90.1% human solved	Mythos 5 89.0% human solved	88.5% human solved	—

在 Frontier-Bench 和 GDPval-AA 这类编码与知识工作评测中，Opus 5 拿下了新的最高分。网络安全任务上，它仍排在 Mythos 5 后面，但差距不大。

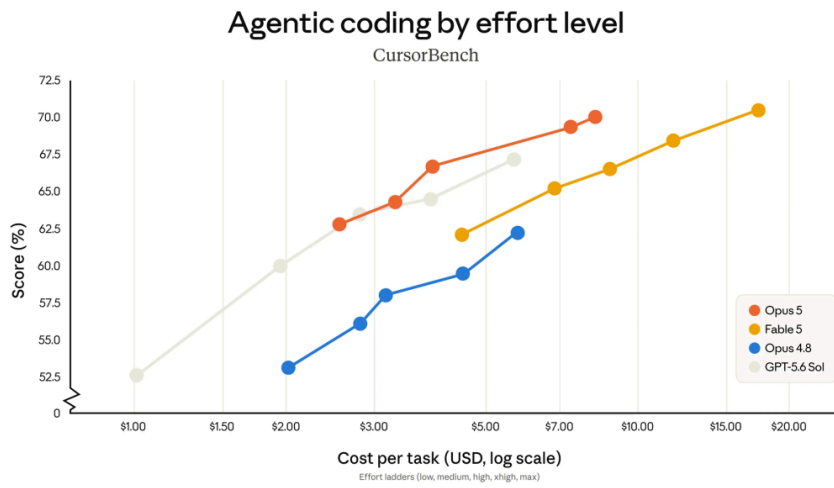
跟上一代 Opus 4.8 比，价格没变，性能大幅跃升。Anthropic 引入了一个 effort 设置，用户可以在智能和 token 消耗之间做取舍，调低就更快更省，拉满就更强。

看几个有意思的评测。

Frontier-Bench v0.1 上，Opus 5 超过所有模型，性能是 Opus 4.8 的2倍以上，单任务成本还更低。

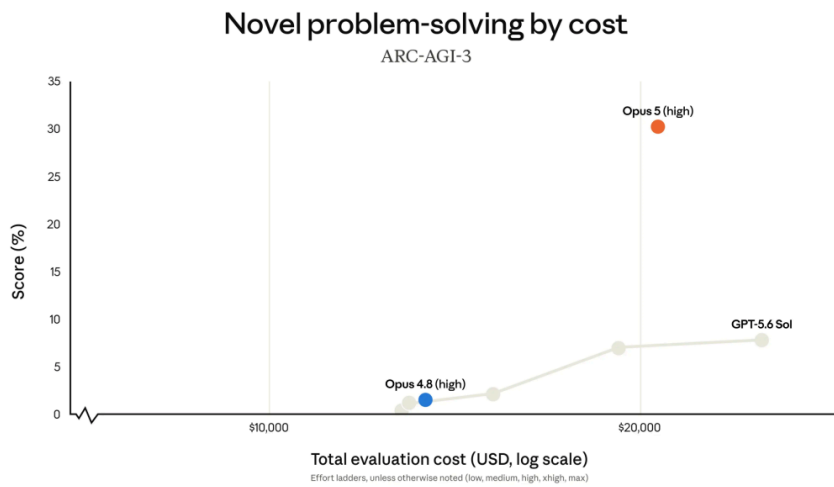


CursorBench 3.2 上，拉到最高 effort，Opus 5 跟 Fable 5 的峰值分差在0.5%以内，单任务成本只有一半。

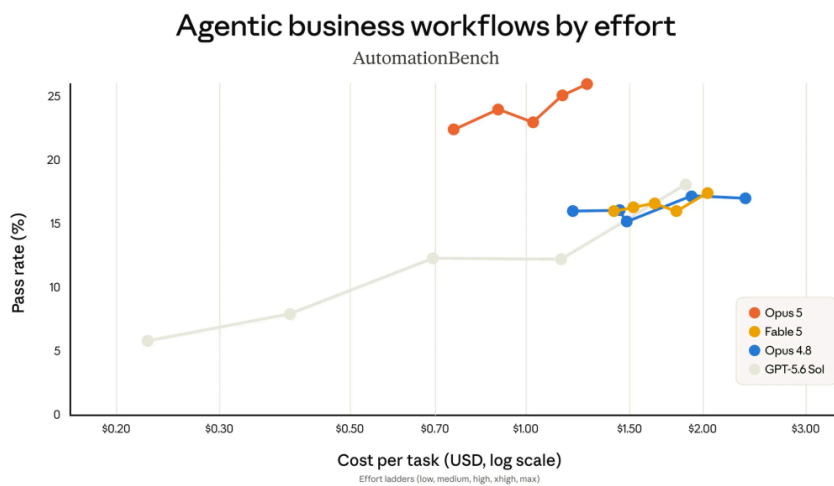


在 high、xhigh、max 三档 effort 下，同等花费能拿到的性能，没有模型比它高。

ARC-AGI 3 考的是解决全新问题的能力，Opus 5 的得分是第二名的3倍。

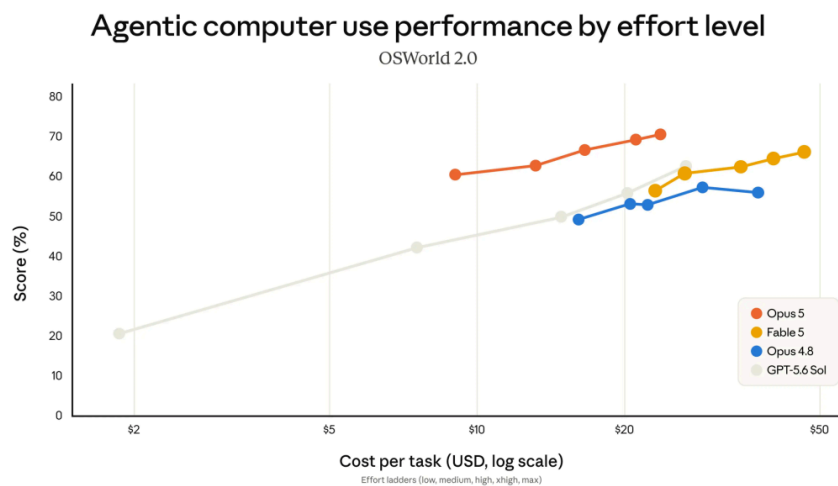


Zapier AutomationBench 测的是从头到尾完成一项业务任务，同等花费下，Opus 5 的通过率大约是第二名的1.5倍。



哪怕把 effort 调到最低，它通过的任务数仍然超过所有其他模型。

OSWorld 2.0 是计算机操作基准，Opus 5 在任何花费水平上都领先，用 Fable 5 最佳成绩三分之一的成本，就超过了 Fable 5 的最好结果。



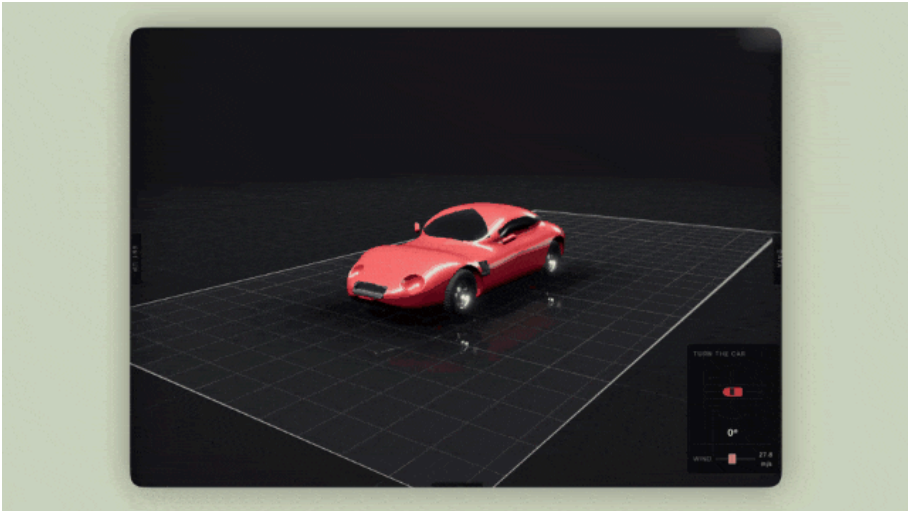
科研方面也有实质进步。

Opus 5 在所有生命科学评测上都优于 Opus 4.8，覆盖结构生物学、有机化学、生物信息学。

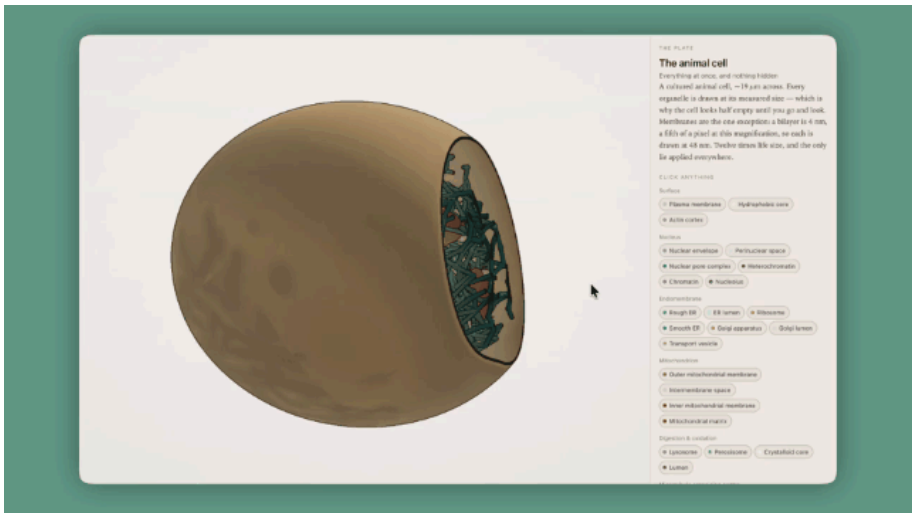
进步最大的两个方向：有机化学里从光谱数据推断分子结构，比 Opus 4.8 高10.2个百分点；蛋白质序列变异对功能影响的预测，高7.7个百分点。

视觉输出能力也明显增强，它能生成质量更高的视觉内容。

例如，可视化空气在流线型（和非流线型）物体上的流动：



或者构建一个 3D 交互式细胞示意图：



80%的提示词，删了

Opus 5 发布的同时，Anthropic 发了一篇关于上下文工程的长文，信息量很大。

你给 Claude 发一条消息，你的 prompt 只是它看到的上下文的一小部分。剩下的来自系统提示词、Skills、CLAUDE.md 文件、记忆模块，以及其他来源。

跟 prompt 不同，上下文是跨请求复用的，没法写得太具体。难点在于：你根本不知道用户下一条消息会问什么。

Anthropic 团队发现，随着 Claude 5 这一代模型能力跃升，他们之前写的大量上下文指导，已经多余了。

Claude Code 的系统提示词，针对 Opus 5 和 Fable 5 这类新模型，删掉80%以上，编码评测分数没有可测量的下降。

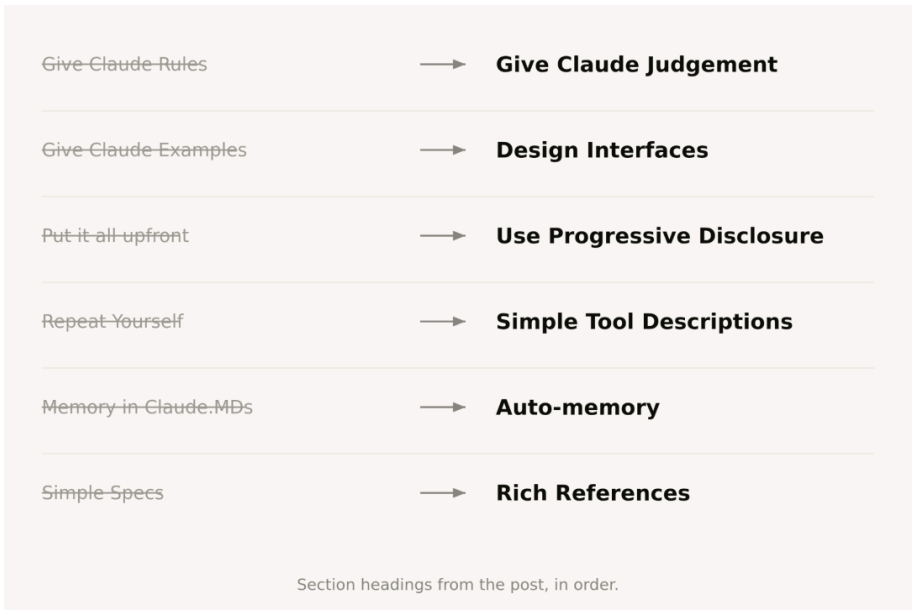
他们翻看了内部使用 Claude Code 的对话记录，发现一个尴尬的现象：同一次请求里，系统提示词说“酌情保留文档”，技能文件说“不要加注释”，用户自己又说“帮我写清楚点”，三条指令互相打架。Claude 能猜出用户真正想要什么，但它得先花精力去调和这些矛盾，再决定怎么做。

这些约束在早期模型上确实必要，能避免最坏情况，比如误删文件。但新模型的判断力已经足够好，很多硬规则反而限制了它。

另外，Claude Code 的工具生态也变了。早期 CLAUDE.md 承担了记忆、信息、指导三重角色。现在有了独立的 memory（记忆）、artifacts（工件）、skills，上下文可以跨会话加载和共享，不必全塞进一个文件。

6条旧经验，全部翻新

Anthropic 把这次认知更新整理成了6组对照。



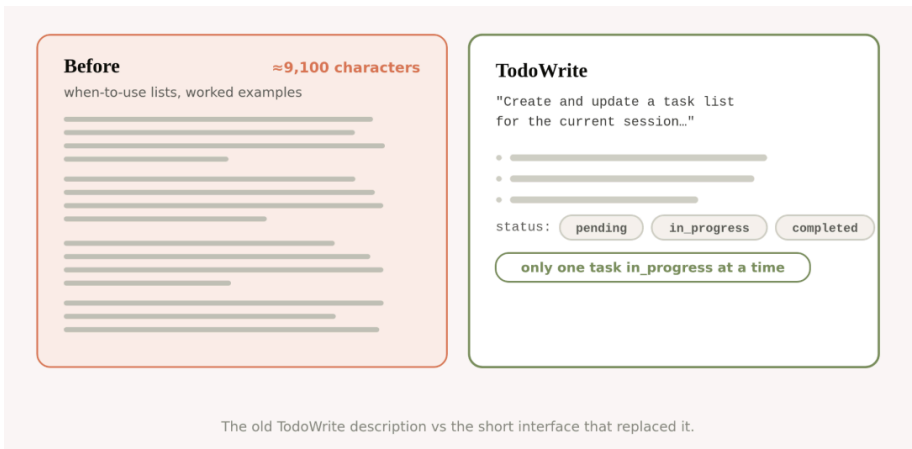
一，从给规则，到给判断空间。

早期 Claude Code 的系统提示词里写过这样的话：代码里默认不写注释，永远不要写多段 docstring 或多行注释块，最多一行短注释；不要创建规划文档、决策文档、分析文档，除非用户明确要求。

这条规则对一部分场景是错的。用户可能有自己的文档偏好，特别复杂的代码确实需要多行注释。但老模型没有规则就会乱写注释，只能接受这个取舍。

新系统提示词换成了一句话：写跟周围代码风格一致的代码，注释密度、命名、惯用法都对齐。

二，从给示例，到设计接口。



以前工具使用的第一法则，是给 Claude 示例，教它怎么调用。新模型上，示例反而把它框死在某个探索空间里。

更好的做法是设计好工具本身的参数和结构。比如一个 Todo 工具，把 status 字段定义为 pending、in_progress、completed 三个枚举值，再加一句“同时只保持一个 in_progress”，Claude 自然就知道该怎么用了。接口设计本身就是最好的指令。

三，从全部前置，到渐进式披露。

Claude Code 早期把代码审查、验证流程的详细说明全写在系统提示词里。不是每次都需要，但需要的时候很关键。

现在 Claude Code 很擅长 progressive disclosure（渐进式披露），在合适的时候加载合适的上下文。验证和代码审查被拆成了独立的 skill，按需调用。

工具层面也用了类似思路。部分工具是延迟加载的，Agent 得先通过 ToolSearch 搜索完整定义，才能使用。这样工具数量可以更多，不占上下文窗口。

同样的原则适用于用户自己的 CLAUDE.md 和 Skill.md。

一个常见误区是把所有可能用到的实践全堆进去，怕 Claude 找不到。更好的做法是建一棵文件树，让 Claude 在需要的时候自己去取。

四，从重复强调，到精简工具描述。

早期模型有时候需要重复指令，或者更容易听从上下文窗口末尾的内容。所以系统提示词里会提到某个工具，工具描述里又写一遍使用说明。

新模型不需要了。重复内容删掉，工具使用说明放在工具描述里，系统提示词不再赘述。

五，从手动记忆，到自动记忆。

以前鼓励用户用 # 快捷键把信息写入 CLAUDE.md，当作 Claude 的记忆。现在 Claude 会自动保存跟当前工作和用户偏好相关的记忆，不用手动操心。

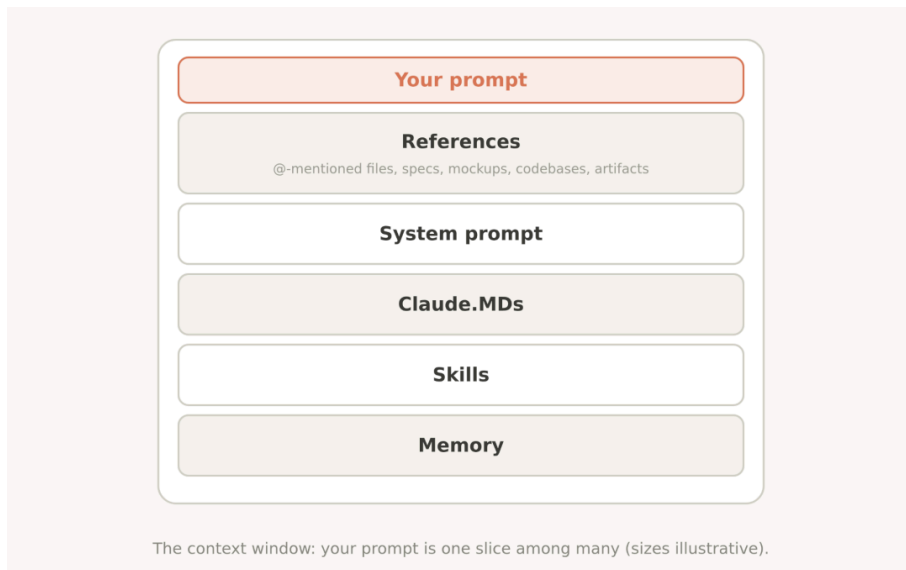
六，从简单规格文档，到丰富参考材料。

Plan mode（规划模式）下，Claude Code 以前重度依赖 markdown 格式的计划文件。跨长项目时，也会在代码库里存 spec（规格说明）供 Claude 参考。

现在 Claude 能处理更复杂的参考形式。HTML 工件可以替代简单的 markdown 文件。代码本身就是最好的 spec，一个详细的测试套件、另一个代码库里待移植的函数，都能当参考。

还有一种叫 rubric（评分标准）的参考形式。它让 Claude 通过动态工作流和验证 Agent 来学习你的审美偏好，比如“什么样的 API 设计算好”。

你的上下文该怎么写



系统提示词跟产品强绑定，告诉 Claude 它在什么产品里运行、要做什么。普通 Claude Code 用户基本不用改。自己搭 Agent harness（智能体框架）的开发者，应该在这里花最多时间。

CLAUDE.md 保持轻量。简单说清楚这个仓库是干什么的，把大部分 token 留给代码库里的坑。比如你的项目把所有类型定义放在一个文件里，别的地方没有，这种信息值得写。不要写 Claude 看一眼文件系统就能知道的东西。

渐进式披露要大胆用。验证流程有好几条独特指令？做成一个 verification skill，在 CLAUDE.md 里引用就行。

Skills 定位是轻量指南，帮 Claude 在需要时找到信息。除非是高风险领域，不要过度约束。长 skill 拆成多个文件。最好的 skill 编码的是你、你的团队、你的产品独有的经验和偏好。

References（参考材料）通过 @ 提及文件来引入。优先用代码形式的参考，保真度最高。一个 HTML 设计稿，通常比一段文字描述或一张截图更能让 Claude 准确还原设计意图。

Anthropic 还上线了一个新命令 claude doctor，在 Claude Code 里输入 /doctor 就能运行。它会自动检查你的 skills 和 CLAUDE.md 文件，帮你做瘦身。

Opus 5 模型能力的跃升，不只是分数涨了几个点，它改变了你跟模型协作的方式。

以前你得当保姆，把规矩写得滴水不漏。现在你得当建筑师，把接口设计好，把信息结构理清楚，然后信任它的判断。

80%的提示词被删掉，模型终于配得上那份信任了。

本文来源：[AIGC开放社区](#)

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。



限时权益申领

* 体验资格每人限申领一次

扫码
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

85 收藏

分享到:

写评论

请发表您的评论

表情 图片

发布评论